// GUIDA GRATIS · LE 6 REPO PER CLAUDE CODE

6 REPO GITHUB, *sei lavori nuovi per Claude Code.*

Le sei repo del reel, una per pagina: cosa fa, chi l'ha scritta, cosa ti serve, i comandi presi dal README ufficiale, il prompt da incollare, cosa devi vedere e i limiti veri. Stelle e licenze lette su GitHub l'11/11/2026. Sono progetti open source di terzi: **non sono nostri** e possono cambiare.

01 · La TV	Clean your TV · Gathin23 · TV Android senza pubblicità	pag. 03
02 · Le previsioni	TimesFM · Google Research · vendite e ordini futuri	pag. 04
03 · Il telefono	ARTEMIS · Google · Claude Code usa un Android vero	pag. 05
04 · I video animati	Lemo-Opuscar · Lemomo · film in 66 stili	pag. 06
05 · Le app	Replica · Jakeschincariol · rifai un'app che paghi	pag. 07
06 · La ricerca	Hyperresearch · Jordan Gibbs · ricerca con fonti controllate	pag. 08
Da dove partire e costi	ordine consigliato, test, cosa è gratis	pag. 09

// COSA TI SERVE

01 Claude Code sul computer
È l'agente che lavora in una cartella del tuo computer: legge il README, lancia i comandi, controlla il risultato. Installazione a pag. 02.

02 Un piano Claude
Claude Code si usa con un abbonamento Claude o con un account API. Tre repo (Lemo-Opuscar, Hyperresearch, ARTEMIS) consumano parecchio: lo trovi scritto nella loro pagina.

03 Il materiale della repo che scegli
Una TV Android, un CSV con le tue vendite, un telefono Android di prova, il nome di un'app o un tema da ricercare.

// COME USARLA

Non servono tutte e sei: **parti dal lavoro che ti pesa di più** e fai i passi della sua pagina in ordine. Il prompt del video basta per partire; sotto trovi i comandi esatti del README, così vedi cosa sta facendo Claude. Se un comando è cambiato, vale il README del link.

// PRIMA DI INIZIARE · LA BASE COMUNE

00 CLAUDE CODE E I TRE MODI *di installare una repo.*

Una repo è una cartella di codice pubblicata su GitHub; il suo **README** è il foglio di istruzioni. Claude Code sa leggerlo ed eseguirlo, ma le sei repo si installano in tre modi diversi: conviene riconoscerli.

// INSTALLA CLAUDE CODE (quickstart ufficiale)

```
curl -fsSL https://claude.ai/install.sh | bash     # macOS, Linux, WSL
irm https://claude.ai/install.ps1 | iex             # Windows PowerShell
claude --version                                    # deve stampare la versione
cd /percorso/della/cartella && claude               # al primo avvio accedi dal browser
```

01 Clona e apri Claude dentro (Clean your TV, ARTEMIS)

`git clone https://github.com/…`, entri nella cartella e lanci `claude`: così Claude usa gli script della repo invece di scriverne di suoi.

02 Plugin dal marketplace (Lemo-Opuscar, Replica, Hyperresearch)

Dentro Claude Code: `/plugin marketplace add owner/repo` poi `/plugin install nome@marketplace`. Le skill compaiono come `/nome:skill`.

03 Pacchetto Python (TimesFM, Hyperresearch)

`pip install …`, meglio in un ambiente virtuale (`python3 -m venv .venv`) per non toccare il Python del sistema.

// PROMPT DI CONTROLLO · DA USARE PRIMA DI OGNI REPO

```
Leggi il README di [link della repo]. Prima di installare qualsiasi cosa dimmi:
1) cosa scarica e dove lo mette, 2) cosa deve già esserci sul mio computer,
3) se servono chiavi API o servizi a pagamento, 4) come si disinstalla.
Poi aspetta il mio ok.
```

// PERCHÉ

Claude Code ti chiede il permesso prima dei comandi che modificano il computer. Con questo prompt sai **prima** cosa approvi: è la differenza tra «ho installato una cosa» e «so cosa ho installato».

Errori comuni — incollare il prompt fuori dalla cartella giusta. Usare `--global` quando il README consiglia l'installazione per progetto. Copiare una chiave API dentro un file che poi pubblichi.

Consiglio — crea una cartella per ogni repo che provi (es. `prove/timesfm/`): se qualcosa va storto, la cancelli e il resto del computer resta pulito.

FONTI · code.claude.com/docs/en/quickstart · code.claude.com/docs/en/discover-plugins

// REPO 01 · LA TV

01 CLEAN YOUR TV: *la TV senza pubblicità.*

github.com/Gathin23/cleanyourtv · 56.235 stelle · licenza MIT. Toglie pubblicità, app preinstallate e tracciamento da una TV **Android TV o Google TV** via Wi-Fi e mette una Home in stile Apple TV. Niente root: le app vengono solo disattivate, non cancellate.



01 Sulla TV
Impostazioni › Preferenze dispositivo › Info › 7 volte OK su «Build» › Opzioni sviluppatore › **Debug USB** attivo (e «Debug di rete/ADB» se c'è).

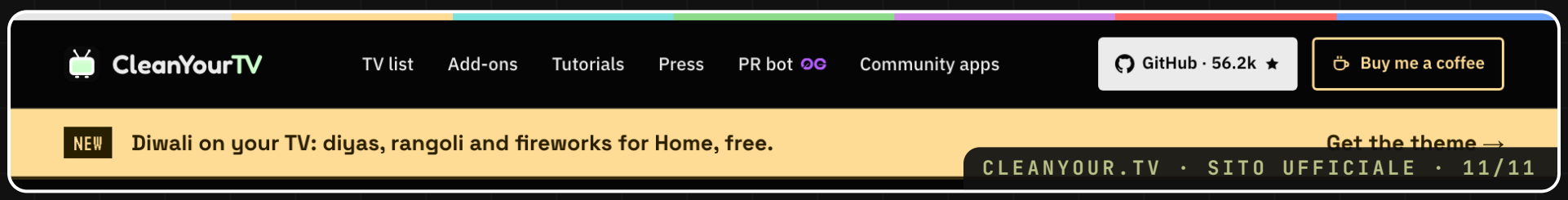
02 Sul computer, stessa Wi-Fi
Servono Python 3, un JDK dalla 17 alla 22 e Android SDK platform 34 con build-tools 34. Su macOS: `brew install openjdk@17 android-commandlinetools`.

03 Clona e apri Claude Code dentro
`git clone https://github.com/Gathin23/cleanyourtv`: così Claude riusa gli script con backup e ripristino.

04 Accetta e rispondi
«Consentire il debug USB?» sulla TV, poi la checklist: app che usi, dock, assistente vocale, Chromecast. Circa 15 minuti.

// IL PROMPT DEL VIDEO · INCOLLALO IN CLAUDE CODE

```
Installa Clean your TV dal README di github.com/Gathin23/cleanyourtv.
La mia TV è questa: [marca e modello].
```



// COSA DEVI VEDERE

Prima di toccare qualcosa Claude salva la lista delle app e le impostazioni, ti chiede conferma su quello che potresti usare e controlla ogni passo con screenshot presi dalla TV. Alla fine: Home nera con il dock delle tue app.

> **Errori comuni —** Samsung, LG, Roku, Vizio e Hisense VIDAA non sono supportati: per loro `cleanyour.tv/not-android` spiega come togliere il tracciamento col telecomando. Fire TV: solo la pulizia, non la nuova Home. Per tornare com'eri: `tools/restore.sh` con la lista applicata.

> **Consiglio —** il sito ufficiale è solo **cleanyour.tv** (cleanyourtv.com e clean-your-tv.net non sono del progetto, dice il README). A lavoro finito spegni il Debug USB.

FONTI · github.com/Gathin23/cleanyourtv · github.com/Gathin23/cleanyourtv/blob/main/PROMPT.md · github.com/Gathin23/cleanyourtv/blob/main/README.it.md · cleanyour.tv/not-android

// REPO 02 · LE PREVISIONI

02 TIMESFM: I TUOI NUMERI, *e cosa succede dopo.*

github.com/google-research/timesfm · 35.397 stelle · Apache-2.0. Modello di Google Research per serie temporali: gli dai lo storico (vendite, ordini, scorte) e ti dà i prossimi valori con il caso migliore e il peggiore. Codice e modelli **fino alla 2.5** sono liberi anche per uso commerciale; i pesi della **3.0** solo non commerciali, e in azienda la 3.0 si usa su BigQuery ML (Google Cloud, a pagamento).



01 Prepara il file
Un CSV con due colonne, periodo e valore, una riga per mese (o settimana), in ordine e senza buchi.

02 Installa (README)
`pip install timesfm[torch]`, oppure `timesfm[mlx]` sui Mac Apple silicon. Al primo avvio scarica il modello `google/timesfm-2.5-200m-pytorch`.

03 Cosa ti restituisce
Claude scrive lo script, lo esegue e ti dà la tabella: previsione, caso peggiore, caso migliore.

// IL PROMPT DEL VIDEO · INCOLLALO IN CLAUDE CODE

```
Installa TimesFM 2.5 dal README di github.com/google-research/timesfm
e fai la previsione del mio file vendite.csv.
```

// ESEMPIO ESEGUITO DA NOI · 11/11/2026 · VENDITE DI ESEMPIO, INVENTATE

36 mesi di vendite (gen 2023 – dic 2025, ultimo mese 1.522) dati a TimesFM 2.5 sul nostro Mac, senza GPU: 198 secondi al primo avvio, compreso lo scaricamento del modello.

mese	previsione	caso peggiore (10%)	caso migliore (90%)
2026-01	1542	1382	1719
2026-02	1617	1432	1820
2026-03	1732	1518	1950
2026-04	1862	1623	2096
2026-05	1954	1703	2202
2026-06	1977	1715	2230

Errori comuni — usare i pesi 3.0 scaricati per l'azienda: la licenza non lo permette. Date mancanti o mischiate. Aspettarsi che sappia di una promozione o di un listino nuovo: vede solo i numeri che gli dai.

Consiglio — prima di fidarti, nascondi gli ultimi 6 mesi veri, fai prevedere quelli dallo storico precedente e confronta: sai quanto sbaglia **sui tuoi numeri**. Se la fascia tra peggiore e migliore è larga, decidi con prudenza.

FONTI · github.com/google-research/timesfm · huggingface.co/google/timesfm-2.5-200m-pytorch · docs.cloud.google.com/bigquery/docs/timesfm-model

// REPO 03 · IL TELEFONO

03 ARTEMIS: CLAUDE CODE *usa un telefono vero.*

github.com/google/artemis · 17.229 stelle · Apache-2.0. Con il suo server MCP Claude Code comanda un telefono **Android** (o un emulatore): apre le app, tocca, scrive, legge lo schermo e i log. È nato per testare app, ma esegue anche compiti in linguaggio naturale.



01 Telefono e computer
Telefono Android con **Debug USB** attivo collegato al computer, oppure un emulatore. Python 3.12+; `start.sh` installa da solo ADB, scrcpy, FFmpeg e le dipendenze.

02 Una chiave API a consumo
Nel file `.env` almeno una tra Gemini, Google, OpenAI, Anthropic, OpenRouter o xAI (`.env.example` della repo).

03 Il pannello
Su `http://localhost:8000`: schermo del telefono in diretta, prompt e replay di ogni compito.

// IL PROMPT DEL VIDEO · INCOLLALO IN CLAUDE CODE

```
Installa ARTEMIS dal README di github.com/google/artemis
e collegalo a Claude Code con il suo server MCP.
```

// COMANDI DAL README

```
git clone https://github.com/google/artemis.git && cd artemis && ./start.sh
uv run artemis mcp --install claude
uv run artemis run "Open Settings, find Battery and tell me current level" --profile flash
```

// COSA DEVI VEDERE

Al primo compito installa sul telefono l'**Artemis Accessibility Helper**: notifica «Artemis test helper is running» e una voce in Impostazioni › Accessibilità. Si toglie con `uv run artemis helper uninstall`. Ogni passo richiede circa 3-5 secondi.

Errori comuni — solo Android, niente iPhone. Ogni compito consuma token sulla chiave API: guarda il costo dopo le prime prove. Telefono bloccato o schermo spento a metà compito.

Consiglio — usa **un telefono di prova** o l'emulatore, mai quello con banca e WhatsApp: l'agente tocca davvero. Sui telefoni personali il README consiglia `ARTEMIS_HELPER_AUTO_INSTALL=false` nel `.env`.

FONTI · github.com/google/artemis · github.com/google/artemis/blob/main/.env.example · github.com/google/artemis/blob/main/mcp_server/README.md

// REPO 04 · I VIDEO ANIMATI

04 LEMO-OPUSCAR: FILM ANIMATI *in 66 stili.*

github.com/lemomo-ai/lemo-opuscar · 14.681 stelle · MIT · di Lemomo. Una libreria di 66 stili di film fatti in codice, dalla lavagna all'acquerello: Claude scrive la storia, disegna ogni fotogramma in codice, compone la musica e fa la voce: niente video generati dall'AI.



01 Prima di iniziare
Node 20+, ffmpeg 7+ e Python 3.11+ (o uv). Il resto lo installa l'agente nella cartella del progetto.

02 Installa il plugin (terminale)
I due comandi qui sotto (marketplace `lemolab`). Al primo film scarica stili e guide in `~/lemo-opuscar`.

03 Chiedi il film
Nomina lo stile in inglese (`chalkboard` = lavagna, `watercolor` = acquerello) o scrivi solo `/lemo-opuscar:lemo-opuscar`. Una tornata di domande (tema, voce o musica tua, storyboard), poi lavora da solo.

// IL PROMPT DEL VIDEO · INCOLLALO IN CLAUDE CODE

```
Installa il plugin Lemo-Opuscar dal README di github.com/lemomo-ai/lemo-opuscar
e fammi un video in stile lavagna sul mio prodotto.
```

Option 1: install the skill (recommended)

In your terminal:

```
claude plugin marketplace add lemomo-ai/lemo-opuscar
claude plugin install lemo-opuscar@lemolab
```

GITHUB · README · INSTALL · 11/11

// COSA DEVI VEDERE

Il film finito nella cartella da cui sei partito: di base 1920×1080 a 24 fps, per i Reel chiedi 1080×1920.

Errori comuni — un film richiede 30-60 minuti e «parecchi token» (README). Gli stili sono tarati su Claude Opus 5.5: con altri modelli possono venire peggio. La voce italiana va provata. Su Ubuntu 24.04 ffmpeg è troppo vecchio.

Consiglio — rispondi **sì** allo storyboard: vedi i fotogrammi chiave nello stile vero prima di spendere un'ora di token, e correggi lì.

FONTI · github.com/lemomo-ai/lemo-opuscar · lemomo-ai.github.io/lemo-opuscar

// REPO 05 · LE APP

05 REPLICA: L'APP CHE PAGHI, *senza i suoi difetti.*

github.com/Jakeschincariol/replica-skill · 8.862 stelle · MIT. Undici skill gratuite, senza account né chiavi, che studiano un'app e te la rifanno: mappa delle schermate, progetto, design, codice, test, lancio. Una legge le recensioni vere e trasforma le lamentele in correzioni. Rifà funzioni e flussi, **mai codice, logo, testi o contenuti** dell'originale.



01 Installa come plugin
Due comandi qui sotto, dentro Claude Code. Le skill diventano `/replica-skill:replica-recon` ecc. Serve Python 3.8+, niente pip.

02 Parti dalla mappa
`replica-recon` legge pagine pubbliche, help center, schede degli store e quello che clicchi nel **tuo** account.

03 Poi in ordine
architect › design › build › backend › test › diff › entrepreneur › brand › launch › deploy. Ognuna legge quello che ha scritto la precedente.

// IL PROMPT DEL VIDEO · INCOLLALO IN CLAUDE CODE

```
Installa le skill di Replica dal README di github.com/Jakeschincariol/replica-skill
e mappami le schermate di [nome dell'app].
```

Or as a plugin, in Claude Code:

```
/plugin marketplace add Jakeschincariol/replica-skill
/plugin install replica-skill@replica-skill
```

GITHUB · README · INSTALL · 11/11

// COSA DEVI VEDERE

Una cartella `replica/` con `recon.md` (schermate, flussi, componenti, modello dei dati) e `features.csv`. Più avanti `replica-diff` ti dà un punteggio di parità e la lista di cosa manca.

Errori comuni — aspettarsi un clone perfetto: il README dice che «un tool di prenotazioni sono settimane». Non entra negli account altrui e non supera i paywall. Prima di vendere: controllo dei marchi e un avvocato.

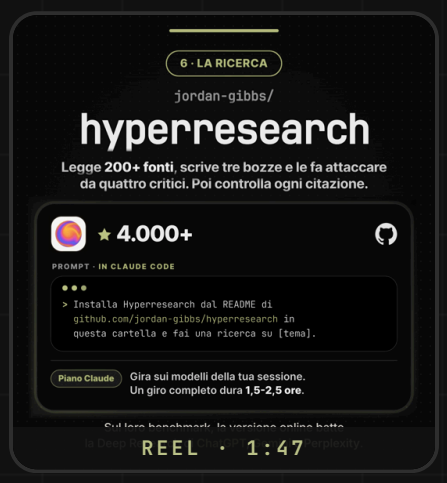
Consiglio — lancia solo `/replica-skill:replica-entrepreneur` sull'app che vuoi rifare: dalle recensioni (ognuna col suo link, le altre le scarta) capisci se vale la pena **prima** di costruire.

FONTI · github.com/Jakeschincariol/replica-skill

// REPO 06 · LA RICERCA

06 HYPERRESEARCH: LA RICERCA *con le fonti controllate.*

github.com/jordan-gibbs/hyperresearch · 4.569 stelle · MIT. Trasforma Claude Code in un agente di ricerca: 16 passi, tre bozze, quattro critici che le attaccano, controllo di ogni citazione prima di consegnare.



01 Installa nella cartella del progetto
Python 3.11+. I tre comandi qui sotto, poi riavvia Claude Code nella stessa cartella.

02 Lancia la ricerca
`/hyperresearch <domanda>`. Domande delimitate: percorso veloce, 30-40 min. Analisi: completo, 1,5-2,5 h.

03 Quante fonti
Profilo base: 55-80. Con `hyperresearch profile use premier`: oltre 250 in tutto, 3-5 h.

// IL PROMPT DEL VIDEO · INCOLLALO IN CLAUDE CODE

```
Installa Hyperresearch dal README di github.com/jordan-gibbs/hyperresearch
in questa cartella e fai una ricerca su [tema].
```

Install

Works in **Claude Code** and **OpenAI Codex**: same pipeline, same vault.

```
cd your-project
pip install hyperresearch

hyperresearch install                    # Claude Code, then: /h
```

GITHUB · README · INSTALL · 11/11

// COSA DEVI VEDERE

Un report finale con le citazioni e una nota per ogni fonte nell'archivio del progetto (`hyperresearch search "parola"`). Se si blocca, `run resume` riparte da dove si era fermato.

Errori comuni — gira sui modelli della tua sessione (sotto-agenti Opus e Sonnet): una ricerca completa consuma molto del piano. Il primo posto su DRACO (77,7) è della **versione online** Hyperresearch Deep, non della repo.

Consiglio — installa per progetto, non con `--global`: dal README, il globale aggiunge ~15 righe a ogni sessione di Claude Code.

FONTI · github.com/jordan-gibbs/hyperresearch · hyperresearch.ai/benchmarks/draco

// DA DOVE PARTIRE · VERIFICA · COSTI

DA DOVE PARTIRE E COME SAI *che funziona.*

L'ordine del reel, con il test che ti dice che la repo sta lavorando davvero e non «sembra a posto».



1 · TimesFM	nascondi 6 mesi veri e confronta la previsione	test
2 · Hyperresearch	apri 5 citazioni a caso: dicono quello che dice il report?	test
3 · Replica	recon.md elenca le schermate che conosci?	test
4 · Lemo-Opuscar	storyboard approvato prima del film	test
5 · ARTEMIS	«dimmi il livello della batteria» sul telefono di prova	test
6 · Clean your TV	Home nuova, poi riavvia la TV: resta?	test

// COSA COSTA (dai README, 11/11/2026)

Clean your TV	MIT, gira tra il tuo computer e la TV	gratis
TimesFM 2.5	Apache-2.0, gira sul tuo computer; la 3.0 in azienda su BigQuery ML	gratis / 3.0 a pagamento
ARTEMIS	Apache-2.0 + chiave API di un modello	a consumo
Lemo-Opuscar	MIT + 30-60 minuti di token per film	piano Claude
Replica	MIT, niente account né chiavi	gratis
Hyperresearch	MIT + 1,5-2,5 h di token per ricerca completa	piano Claude

01 **Una repo alla volta**
Installa, fai il test, usala una settimana. Solo dopo passa alla successiva: se qualcosa si rompe, sai da dove viene.

02 **Scrivi cosa hai approvato**
In una nota della cartella: comandi lanciati, chiavi usate, come si disinstalla. Ti serve il giorno che vuoi toglierla.

> **Consiglio —** le tre che consumano token (Lemo-Opuscar, Hyperresearch, ARTEMIS) provale su un compito piccolo e guarda il consumo, prima di lanciarle su quello vero.

// LINK UFFICIALI

TUTTI I LINK, IN *un posto.*

Repo e pagine ufficiali citate nella guida, aperte e controllate l'11/11/2026.

Claude Code · installazione	code.claude.com/docs/en/quickstart	00
Claude Code · plugin	code.claude.com/docs/en/discover-plugins	00
Clean your TV · repo	github.com/Gathin23/cleanyourtv	01
Clean your TV · guida in italiano	github.com/Gathin23/cleanyourtv/blob/main/README.it.md	01
Clean your TV · sito ufficiale	cleanyour.tv	01
Clean your TV · TV non Android	cleanyour.tv/not-android	01
TimesFM · repo	github.com/google-research/timesfm	02
TimesFM 2.5 · modello	huggingface.co/google/timesfm-2.5-200m-pytorch	02
TimesFM su BigQuery ML	docs.cloud.google.com/bigquery/docs/timesfm-model	02
ARTEMIS · repo	github.com/google/artemis	03
ARTEMIS · server MCP	github.com/google/artemis/blob/main/mcp_server/README.md	03
Lemo-Opuscar · repo	github.com/lemomo-ai/lemo-opuscar	04
Lemo-Opuscar · galleria stili	lemomo-ai.github.io/lemo-opuscar	04
Replica · repo	github.com/Jakeschincariol/replica-skill	05
Hyperresearch · repo	github.com/jordan-gibbs/hyperresearch	06
Hyperresearch · benchmark DRACO	hyperresearch.ai/benchmarks/draco	06

// RICORDA

Sono sei progetti open source di persone e team diversi, **non prodotti nostri**: li abbiamo letti e provati dove si poteva, ma cambiano in fretta. Prima di installare, fai leggere il README a Claude con il prompt di controllo di pag. 02. Telefono di prova per ARTEMIS, backup per la TV, numeri veri per TimesFM.

Stelle, comandi e licenze sono quelli dell'11/11/2026: se qualcosa non corrisponde, vale il README del link.